



DS Guardian - Manual Completo de Usuario y Referencia Técnica

AI Data Science Governance & Quality Assurance System

DS Guardian

DS Guardian es un framework de código abierto en Python diseñado para ayudar a los profesionales de la Ciencia de Datos a construir flujos de trabajo de Machine Learning más robustos, reproducibles y confiables. Se enfoca en prevenir errores comunes como la fuga de datos (*data leakage*), manejar conjuntos de datos del mundo real y proporcionar verificaciones de calidad automatizadas antes del entrenamiento del modelo.



Why DS Guardian?

Este proyecto nació de una necesidad real observada en el desarrollo y enseñanza de la Ciencia de Datos:

1. **Evitar repetir código en notebooks:** La experimentación interactiva suele llevar a celdas desordenadas y fragmentadas. DS Guardian abstrae los componentes repetitivos en funciones modulares limpias.
 2. **Aplicar buenas prácticas desde el inicio:** Establece una base sólida previniendo errores comunes de ingeniería antes de que los datos toquen los modelos de Machine Learning.
 3. **Facilitar el aprendizaje sin sacrificar calidad:** Diseñado tanto para profesionales que buscan robustecer sus flujos como para estudiantes que desean aprender a desarrollar código limpio y modular bajo estándares de nivel de producción.
-

Filosofía de Funcionamiento

DS Guardian actúa como una capa de auditoría y control de calidad entre la ingesta de datos y el entrenamiento del modelo:

```
graph LR
  RawData["RawData[\"Datos Crudos (Sucios)\"]"] --> EDA["1. EDA & Memory  
Optimization<br>(eda.py)"]
  EDA --> Partition["2. Train/Test Split"]
  Partition --> Cleaning["3. Preprocesamiento Seguro<br>(limpieza.py)"]
  Cleaning --> Audit["4. Agente QA Auditor<br>(auditoria.py)"]
  Audit -- "Fallo (Data Leakage, Nulos, etc.)" --> Cleaning
  Audit -- "Éxito (Verificación)" --> Modeling["5. Entrenamiento &  
Ajuste<br>(modelos.py)"]
```

Navegación Rápida

Comienza a explorar las diferentes secciones:

- [Guía de Inicio](#): Configuración inicial y creación de tu primer pipeline.
- [Instalación](#): Requisitos y opciones de instalación del paquete.
- [Arquitectura](#): Detalles sobre el flujo de control y mitigación del Data Leakage.
- [Referencia de la API](#): Métodos, firmas y docstrings detallados de los módulos.
- [Tutoriales](#): Guías paso a paso de cada módulo.
- [Preguntas Frecuentes](#): Solución a dudas comunes del framework.

Guía de Inicio Rápido

Esta guía te ayudará a ejecutar tu primer flujo de trabajo auditado de Machine Learning con **DS Guardian** en pocos minutos.

1. Instalación Rápida

Puedes instalar el framework de manera local y editable navegando al directorio del repositorio

y ejecutando:

```
pip install -e .
```

O instalando las dependencias directamente:

```
pip install -r requirements.txt
```

Para más detalles sobre la instalación y el entorno, consulta la página de [Instalación](#).

2. Creación del script `ejemplo.py`

Aquí tienes un pipeline completo que muestra cómo preprocesar de forma segura, auditar los datos y entrenar un clasificador RandomForest:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from ds_guardian import eda, limpieza, modelos, auditoria

# 1. Carga de datos de ejemplo (con nulos y outliers)
np.random.seed(42)
df = pd.DataFrame({
    'Edad': [25, 30, np.nan, 45, 50, 55, 60, 65, 70, 75],
    'Salario': [50000, 60000, 70000, 80000, 90000, 100000, 110000, 120000, 600000,
-100000],
    'Target': [1, 0, 1, 0, 1, 0, 1, 0, 1, 1]
})

# 2. Exploración y Capping de Outliers
df = eda.acotar_outliers_iqr(df, columnas=['Salario'])

# 3. División Train/Test antes de limpiar para evitar Data Leakage
X = df.drop('Target', axis=1)
y = df['Target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# 4. Preprocesamiento aislado y seguro
X_train, X_test = limpieza.imputar_nulos(X_train, X_test)
X_train, X_test = limpieza.escalar_caracteristicas(X_train, X_test,
```

```
metodo='standard')  
  
# 5. Auditoría QA  
if auditoria.revisar_datos_finales(X_train, y=y_train):  
    # Entrenar modelo  
    modelo = RandomForestClassifier(random_state=42)  
    modelo.fit(X_train, y_train)  
  
    # Predecir y evaluar  
    y_pred = modelo.predict(X_test)  
    modelos.evaluar_clasificacion(y_test, y_pred,  
save_path='plots/confusion_matrix.png')
```

3. Ejecutar y Observar

Corre el script en tu terminal:

```
python ejemplo.py
```

- Verás en la consola la salida coloreada del módulo de auditoría de datos.
- Se generará el gráfico de la matriz de confusión en `plots/confusion_matrix.png`.

Guía de Instalación

Este documento detalla los requisitos del sistema y las diferentes opciones para instalar **DS Guardian** de forma portable en cualquier entorno.

Requisitos de Entorno

- **Versión de Python:** Python 3.9 o superior.
- **Dependencias Core:**
 - `pandas` $\geq$ 1.5.0
 - `numpy` $\geq$ 1.20.0
 - `scikit-learn` $\geq$ 1.0.0
 - `seaborn` $\geq$ 0.12.0

- `matplotlib` `>= 3.5.0`
 - `python-docx` `>= 1.0.0`
-



Quick Start (Inicio Rápido)

Para descargar, instalar el framework e iniciar la documentación local en un entorno limpio:

```
# 1. Clonar el repositorio
git clone https://github.com/holahola144/ds_guardian.git

# 2. Entrar a la carpeta del proyecto
cd ds_guardian

# 3. Instalar en modo editable
pip install -e .

# 4. Iniciar el servidor local de documentación
mkdocs serve
```

Luego, abre en tu navegador la dirección: `http://127.0.0.1:8000`

Opción 1: Instalación de Dependencias desde `requirements.txt`

Si prefieres no instalar el framework como paquete en el sistema, sino simplemente importar la subcarpeta `ds_guardian/` en tus propios scripts, puedes instalar únicamente sus dependencias:

```
pip install -r requirements.txt
```

[!NOTE] Si utilizas **Conda** o un entorno virtual (`venv`, `poetry`, etc.), asegúrate de activarlo en tu terminal antes de ejecutar `pip install` o `mkdocs serve`.

Verificación de la Instalación

Para verificar que la librería se ha instalado correctamente y no hay conflictos de importación, ejecuta:

```
python prueba_robusta.py
```

Si la consola imprime el mensaje de finalización exitosa y no arroja errores de importación, la instalación ha concluido con éxito.

Tutorial: Auditoría de Datos e Historial de Modelos

El módulo `ds_guardian.auditoria` funciona como un agente QA automatizado que valida la sanidad de tus datasets finales y guarda registros históricos de rendimiento.

1. Auditoría del Dataset Final

Antes de pasar un DataFrame de características al método `.fit()` del estimador de Machine Learning, es vital validar que esté limpio para evitar caídas catastróficas en tiempo de ejecución.

`revisar_datos_finales` analiza de manera estática y dinámica el DataFrame y arroja: *

ERRORES FATALES (Rojo): Si detecta variables de texto no codificadas, valores nulos remanentes o una correlación perfecta (>0.99) con la variable objetivo (**Data Leakage**).

Retorna `False` (o dict con `'aprobado': False`). * **ADVERTENCIAS (Amarillo):** Si detecta multicolinealidad entre características numéricas (>0.95) o clases fuertemente desbalanceadas ($>65\%$ clase mayoritaria). * **OK (Verde):** Si las verificaciones básicas son exitosas. Retorna `True` (o dict con `'aprobado': True`).

```
from ds_guardian import auditoria
```

```
# Ejecutar auditoría de sanidad obteniendo el detalle completo
```

```
detalle = auditoria.revisar_datos_finales(X_train_scaled, y=y_train,
```

```
retornar_detalle=True)

if detalle['aprobado']:
    print("El dataset es seguro para el modelo.")
else:
    print("Corrige las inconsistencias señaladas en la consola.")
```

2. Generación de Reportes Ejecutivos en Markdown

Puedes generar un reporte completo en formato Markdown listo para presentar o guardar en la documentación de tu proyecto:

```
# Generar reporte técnico Markdown
auditoria.generar_reporte_auditoria_markdown(
    nombre_proyecto="Clasificación de Clientes Churn",
    df_info={
        'filas': len(X_train),
        'columnas': X_train.shape[1],
        'nulos': X_train.isnull().sum().sum(),
        'memoria_mb': round(X_train.memory_usage().sum() / 1024 / 1024, 2)
    },
    resultado_auditoria=detalle['aprobado'],
    metricas_modelo={'Accuracy': 0.92, 'F1-Score': 0.89, 'ROC-AUC': 0.94},
    top_features=top_features_df,
    output_path="reports/reporte_auditoria_churn.md",
    detalle_auditoria=detalle
)
```

3. Historial Comparativo de Experimentos

Permite llevar un registro de las métricas obtenidas por tus modelos a lo largo del desarrollo. Si una nueva ejecución del modelo mejora o empeora los resultados históricos del proyecto, el framework te lo notificará:

```
# Guarda la métrica del experimento en 'historial_proyectos.json'
auditoria.registrar_y_comparar_modelo(
    nombre_proyecto='Clasificacion_Clientes',
```

```
metrica_principal_nombre='F1-Score',  
valor_metricea=0.86,  
maximizar=True      # True si un valor mayor es mejor (F1, Accuracy), False si  
es un error (MSE)  
)
```

Tutorial: Análisis Exploratorio de Datos (EDA)

El módulo `ds_guardian.eda` proporciona herramientas útiles para comprender la salud de tus datos y optimizar el consumo de recursos de computación.

1. Optimización del Consumo de Memoria RAM

Cuando trabajas con DataFrames de gran escala, Pandas suele asignar tipos de datos excesivamente grandes por defecto (como `int64` o `float64`). El método `optimizar_memoria` analiza los rangos de valores numéricos de las columnas y realiza un downcast seguro (ej: a `int8` o `float32`), disminuyendo la memoria consumida hasta en un 70%.

```
import pandas as pd  
from ds_guardian import eda  
  
# Cargar tu dataset  
df = pd.read_csv('datos_gigantes.csv')  
  
# Optimizar de forma automática  
df_optimizado = eda.optimizar_memoria(df)
```

2. Detección y Tratamiento de Outliers (Winsorization)

La presencia de outliers (valores atípicos) puede distorsionar fuertemente las predicciones de los modelos basados en distancias (como regresión lineal o SVM).

Detección con el Rango Intercuartílico (IQR):

```
# Reporta el número de outliers identificados en el DataFrame por columna
eda.detectar_outliers_iqr(df)
```

Winsorization (Capping sin Data Leakage):

En lugar de descartar filas valiosas eliminando outliers, limitamos (acotamos) los valores extremos reemplazándolos con los límites mínimo y máximo definidos por el Rango Intercuartílico.

Para evitar **Data Leakage**, los límites del IQR deben calcularse exclusivamente sobre el conjunto de entrenamiento (train) y aplicarse al de prueba (test):

```
# Acota los valores extremos en train y test sin contaminar los límites de train
X_train_canned, X_test_capped = eda.acotar_outliers_iqr(
    df=X_train,
    columnas=['Ingresos', 'Edad'],
    df_test=X_test
)
```

Tutorial: Limpieza Segura (Cleaning)

El módulo `ds_guardian.limpieza` se encarga de preparar los datos numéricos y categóricos para que el estimador no falle, asegurando un aislamiento total entre entrenamiento y validación.

1. Imputación de Valores Faltantes (Nulos)

Para evitar la fuga de datos (Data Leakage), calculamos las estadísticas de imputación (como la mediana o la moda) **únicamente** sobre los datos de entrenamiento y aplicamos esos mismos valores fijos a los conjuntos de prueba:

```
from ds_guardian import limpieza

# Imputación segura sin fugas
```

```
X_train_clean, X_test_clean = limpieza.imputar_nulos(  
    X_train,  
    X_test,  
    estrategia_num='median',      # 'mean' o 'median'  
    estrategia_cat='most_frequent' # 'most frequent'  
)
```

2. Codificación Categórica Alineada

Si aplicamos One-Hot Encoding por separado sobre el conjunto de entrenamiento y de validación, es altamente probable que el número y orden de columnas resultantes difieran (debido a categorías raras presentes solo en un subconjunto).

`codificar_variables` previene esto de forma interna, alineando automáticamente las dimensiones finales:

```
# Codifica categóricas y asegura simetría de columnas en Train y Test  
X_train_enc, X_test_enc = limpieza.codificar_variables(X_train_clean, X_test_clean)
```

3. Escalamiento Seguro de Características

Escalamos variables numéricas preservando las estadísticas descriptivas estrictamente del conjunto de entrenamiento:

```
# Escalar a media 0 y varianza 1 (Standard) o rango [0,1] (MinMax)  
X_train_scaled, X_test_scaled = limpieza.escalar_caracteristicas(  
    X_train_enc,  
    X_test_enc,  
    metodo='standard'  
)
```

Tutorial: Evaluación y Optimización de Modelos

El módulo `ds_guardian.modelos` ofrece envoltorios optimizados para medir la calidad real de tus predictores y buscar hiperparámetros.

1. Ajuste de Hiperparámetros por Validación Cruzada

Busca la combinación óptima de parámetros para tu estimador evitando el sobreajuste (overfitting):

```
from ds_guardian import modelos
from sklearn.ensemble import RandomForestClassifier

# Definir la grilla de búsqueda
grilla_params = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10]
}

# Ejecutar optimización aleatoria estructurada
mejor_modelo = modelos.optimizar_hiperparametros(
    RandomForestClassifier(random_state=42),
    param_grid=grilla_params,
    X=X_train_scaled,
    y=y_train,
    cv=5,                # Pliegues de validación cruzada
    n_iter=10,           # Intentos de combinaciones
    scoring='f1'
)
```

2. Evaluación Detallada de Predicciones

Evaluación de Clasificación:

Genera un reporte consolidando Precision, Recall, F1-Score y Accuracy, guardando además una matriz de confusión interactiva:

```
# Evaluar y guardar matriz de confusión
modelos.evaluar_clasificacion(
    y_true=y_test,
    y_pred=y_pred,
    y_prob=y_prob,
    save_path='plots/matriz_confusion.png'
)
```

Evaluación de Regresión:

Calcula e imprime métricas estándar de error absoluto, cuadrático y coeficiente de determinación:

```
# Evaluar regresores continuos
modelos.evaluar_regresion(y_true=y_test, y_pred=y_pred)
```

Tutorial: Visualización Premium

El módulo `ds_guardian.visualizacion` permite generar reportes gráficos limpios y modernos, ideales para presentaciones técnicas o anexos de ingeniería.

1. Configuración Global de Estilos

El framework soporta temas visuales claro (`light`) y oscuro (`dark`) de forma automática:

```
from ds_guardian import visualizacion

# Configurar el estilo visual por defecto del notebook o script
visualizacion.configurar_estilo(theme='dark')
```

2. Generación de Gráficos No Bloqueantes (Headless)

Si estás ejecutando el pipeline en un servidor remoto o un script automatizado en segundo plano, llamar a gráficos interactivos detendrá la ejecución del programa. Todas las funciones de visualización de **DS Guardian** admiten el argumento `save_path` para evitar esto:

```
# Grafica y almacena directamente a disco sin abrir ventanas interactivas
visualizacion.plot_distribucion(df, columna='Edad', save_path='plots/edad_dist.png')

# Heatmap de correlación lineal
visualizacion.plot_correlacion(df, save_path='plots/matriz_correlacion.png')
```

3. Importancia de Variables en Modelos

Permite graficar de manera ordenada y estilizada qué variables del dataset tuvieron mayor peso en la toma de decisión del algoritmo entrenado:

```
# Grafica las características más relevantes para el estimador
visualizacion.plot_importancia_caracteristicas(
    modelo_entrenado,
    feature_names=list(X_train.columns),
    max_features=10,
    save_path='plots/feature_importances.png'
)
```

Referencia de la API (API Reference)

Este documento contiene la firma y descripción de las principales funciones expuestas por **DS Guardian**.

Módulo: `ds_guardian.eda`

`resumir_datos(df)`

Imprime dimensiones, tipos de datos y estadísticas descriptivas de forma segura controlando datasets vacíos.

`missing_values_table(df)`

Calcula y formatea una tabla con el recuento y porcentaje de nulos por columna.

`optimizar_memoria(df)`

Optimiza los tipos de datos de las columnas numéricas (ej. downcast a `int8` o `float32`) para reducir el consumo en RAM.

`detectar_outliers_iqr(df)`

Detecta y reporta outliers en variables numéricas usando el Rango Intercuartílico (IQR).

`acotar_outliers_iqr(df, columnas=None, factor=1.5, df_test=None)`

Aplica Winsorization acotando valores extremos dentro de los límites del IQR sin eliminar filas. Si se pasa `df_test`, los límites del IQR se calculan únicamente con `df` (train) y se aplican defensivamente sobre `df_test` evitando data leakage.

Módulo: `ds_guardian.limpieza`

`imputar_nulos(df_train, df_test=None, estrategia_num='median', estrategia_cat='most_frequent')`

Imputa nulos en características numéricas y categóricas evitando Data Leakage.

tratar_duplicados(df)

Busca y remueve filas duplicadas en el DataFrame.

codificar_variables(df_train, df_test=None)

Aplica One-Hot Encoding a variables categóricas alineando el número de columnas resultantes entre Train y Test.

escalar_caracteristicas(df_train, df_test=None, metodo='standard', columnas=None)

Escala variables numéricas usando `StandardScaler` o `MinMaxScaler` aislando correctamente el conjunto de prueba.

Módulo: ds_guardian.visualizacion

configurar_estilo(theme='light')

Configura el estilo visual global de seaborn y matplotlib. Soporta temas `light` (claro) y `dark` (oscuro).

plot_distribucion(df, columna, save_path=None)

Grafica histograma y curva KDE de una variable. Si se pasa `save_path`, la imagen se guarda en disco y no bloquea el proceso.

plot_categorica(df, columna, max_categorias=20, save_path=None)

Gráfico de barras horizontales para variables categóricas.

plot_correlacion(df, save_path=None)

Heatmap de correlación lineal sobre variables numéricas.

```
plot_importancia_caracteristicas(modelo, feature_names,  
max_features=15, save_path=None)
```

Grafica el orden de importancia de variables del estimador (soporta modelos basados en árboles y modelos lineales con coeficientes).

Módulo: `ds_guardian.modelos`

```
evaluar_clasificacion(y_true, y_pred, y_prob=None,  
save_path=None)
```

Reporte detallado de clasificación, Accuracy, ROC-AUC y heatmap de la matriz de confusión.

```
evaluar_regresion(y_true, y_pred)
```

Calcula e imprime MSE, RMSE, MAE y R2 Score.

```
graficar_importancia_caracteristicas(modelo, feature_names,  
max_features=15, save_path=None)
```

Extrae y visualiza la importancia relativa de las características del modelo.

```
validacion_cruzada(modelo, X, y, cv=5, scoring='accuracy')
```

Prueba el modelo mediante K-Fold Cross Validation informando medias y desviaciones estándar.

```
optimizar_hiperparametros(modelo, param_grid, X, y, cv=3,  
n_iter=10, scoring='accuracy')
```

Optimización aleatoria de hiperparámetros con RandomizedSearchCV.

Módulo: `ds_guardian.auditoria`

`revisar_datos_finales(df, y=None, retornar_detalle=False)`

Agente QA que ejecuta un checklist de calidad en el dataset e informa advertencias/errores con colores en la consola. Si `retornar_detalle=True`, devuelve un diccionario completo con el estado de cada validación individual.

`generar_reporte_auditoria_markdown(nombre_proyecto, df_info, resultado_auditoria, metricas_modelo, top_features=None, output_path="reporte_auditoria.md", detalle_auditoria=None)`

Genera un informe técnico completo en formato Markdown (HTML/GitHub compatible) con los resultados de la auditoría QA, calidad de datos y evaluación del modelo.

`registrar_y_comparar_modelo(nombre_proyecto, metrica_principal_nombre, valor_metrica, maximizar=True)`

Guarda el desempeño del modelo en `historial_proyectos.json` y alerta si mejoró o empeoró respecto a ejecuciones previas.
